



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251211
Nama Lengkap	Jendri Mansamnaf Japen
Minggu ke / Materi	01 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 13.1

13.1.1 Tuple Immutable

Tuple bisa dikatakan menyerupai list. Nilai yang disimpan dalam tuple dapat berupa apapun dan diberikan indeks bilangan bulat (integer). Perbedaan utama adalah sifat tuple yang immutable, yaitu dimana anggota dalam tuple tidak bisa dirubah. Tuple sendiri dapat dibandingkan (compare) dan bersifat hashable sehingga dapat dimasukkan dalam list sebagai key pada dictionary python.

Sebuah objek disebut hashable jika memiliki nilai hash yang tidak pernah berubah selama hidupnya (menggunakan method `__hash__()`), dan dapat dibandingkan dengan benda-benda lain (menggunakan method `__eq__()` atau `__cm__()`). Obyek Hashable membandingkan yang sama dengan memiliki nilai hash yang harus sama. Hashability membuat sebuah objek yang dapat digunakan sebagai kamus kunci dan mengatur anggota, karena struktur data ini menggunakan nilai hash secara internal. Objek pada python built-in biasanya hashable, sementara tidak bisa berubah wadah (seperti daftar atau kamus) adalah. Objek yang merupakan instance dari kelas yang didefinisikan pengguna yang hashable secara default; mereka semua membandingkan yang tidak sama, dan mereka nilai hash adalah id().

selengkapnya : <https://docs.python.org/3/glossary.html>

Sintaks penulisan tuple :

```
>>> t = 'a','b','c','d','e'
```

Sintaks lain penulisan tuple dapat menggunakan tanda kurung:

```
>>> t = ('a','b','c','d','e')
```

Tuple dengan satu element, ditambahkan koma (,) dibelakang penulisan tuple.

```
>>> t1 = ('a',)
>>> type(t1)
<type 'tuple'>
```

Jika tidak menambahkan tanda koma, akan dianggap sebagai string.

```
>>> t2 = ('a')
>>> type(t2)
<type 'str'>
```

Model sintaks lain dengan menggunakan fungsi built-in tuple. Ketika tidak diisi dengan argument apapun, secara langsung akan membentuk tuple kosong.

```
>>> t = tuple()
>>> print(t)
()
```

Jika argumennya berupa urutan (string, list, atau tuple), akan mengembalikan nilai tuple dengan elemen-elemen yang berurutan.

```
>>> t = tuple('dutarawacana')
>>> print(t)
('d', 'u', 't', 'a', 'r', 'a', 'w', 'a', 'c', 'n', 'a')
```

Tuple merupakan nama dari constructor, kita tidak bisa menggunakannya sebagai nama variabel. Sebagian besar operator yang bekerja pada list bekerja juga pada tuple. Tanda kurung kotak [] menandakan indeks element dari tuple.

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> print(t[0])
'a'
```

Untuk menampilkan rentang nilai dari element tuple dapat menggunakan:

```
>>> print(t[1:3])
('b', 'c')
```

Tuple bersifat immutable artinya element pada tuple tidak dapat berubah. Jika anda berusaha mengubah tuple, maka akan didapatkan error

```
>>> t[0] = 'A'
TypeError: object doesn't support item assignment
```

Element pada tuple tidak dapat dirubah tetapi dapat diganti dengan elemen lain.

```
>>> t = ('A',) + t[1:]
>>> print(t)
('A', 'b', 'c', 'd', 'e')
```

13.1.2 Membandingkan Tuple

Operator perbandingan (compare) dapat bekerja pada tuple dan model sekuensial lainnya (list, dictionary, set). Cara kerjanya dengan membandingkan elemen pertama dari setiap sekuensial yang ada. Jika ditemukan adanya kesamaan, akan berlanjut ke elemen berikutnya. Proses ini berlangsung terus menerus hingga ditemukan adanya perbedaan.

```
>>> (0, 1, 2) < (0, 3, 4)
True
>>> (0, 1, 2000000) < (0, 3, 4)
True
```

Fungsi (sort) pada python bekerja dengan cara yang sama. Tahap pertama akan melakukan pengurutan berdasarkan elemen pertama. Pada kasus tertentu akan mengurutkan berdasarkan elemen kedua dan sebagainya. Fitur ini disebut dengan DSU – (Decorate, Sort, Undercorate)

1. Decorate – urutan (sekuensial) membangun daftar tuple dengan satu atau lebih key pengurutan sebelum elemen dari urutan (sekuensial).
2. Sort – list tuple menggunakan sort (fungsi bawaan di python)
3. Undercorate – melakukan ekstraksi pada elemen yang telah diurutkan pada satu sekuensial.

Untuk memahaminya, silakan perhatikan contoh dibawah ini. Kita mempunyai sebuah kalimat dan akan melakukan pengurutan kata dalam kalimat tersebut dari yang paling panjang ke yang paling pendek.

```
1
2 kalimat = 'but soft what light in yonder window breaks'
3 dafkata = kalimat.split()
4 t = list()
5 for kata in dafkata:
6     t.append((len(kata), kata))
7
8 t.sort(reverse=True)
9
10 urutan = list()
11 for length, kata in t:
12     urutan.append(kata)
13
14 print(urutan)
15
16
```

Looping yang pertama akan membuat daftar tuple, yang berisi daftar kata sesuai dengan panjangnya. Fungsi sort akan membandingkan elemen pertama dari list dari panjang kata yang ada dan kemudian akan menuju ke elemen kedua jika kondisi sesuai. Keyword reverse=True digunakan untuk melakukan urutan secara terbalik (descending). Looping kedua akan membangun daftar tuple dalam sesuai urutan alfabet diurutkan berdasarkan panjangnya. Pada contoh diatas Empat kata dalam kalimat akan diurutkan secara alfabet terbalik. Kata "what" akan muncul sebelum "soft" didalam list.

Output program diatas dapat dilihat dibawah ini:

```
['yonder', 'window', 'breaks', 'light', 'what',  
'soft', 'but', 'in']
```

Kata-kata yang muncul diurutkan sebagai list dengan urutan panjang kata dari yang paling panjang ke kata yang paling pendek.

13.1.3 Penugasan Tuple

Salah satu fitur unik dari Python adalah kemampuannya untuk memiliki tuple disisi kiri dari statement penugasan. Hal ini mengijinkan untuk menetapkan lebih dari satu variabel pada sisi sebelah kiri secara berurutan. Contohnya ketika ada dua daftar elemen yang merupakan urutan. Kita akan mencoba menetapkan elemen pertama dan kedua dari urutan tersebut kedalam variabel x dan y dalam satu statement.

```
>>> m = [ 'have', 'fun' ]  
>>> x, y = m  
>>> x  
'have'  
>>> y  
'fun'  
>>>
```

Python akan menterjemahkan sintaks tuple dalam langkah-langkah sebagai berikut:

```
>>> m = [ 'have', 'fun' ]  
>>> x = m[0]  
>>> y = m[1]  
>>> x  
'have'  
>>> y  
'fun'  
>>>
```

Tuple yang digunakan pada contoh diatas menggunakan statement penugasan disebelah kiri dan tidak menggunakan tanda kurung. Berikut ini merupakan contoh yang sama dengan menggunakan tanda kurung (parentheses).

```
>>> m = [ 'have', 'fun' ]  
>>> (x, y) = m  
>>> x  
'have'  
>>> y  
'fun'  
>>>
```

Tuple memungkinkan pula untuk menukar nilai variabel dalam satu statement.

```
>>>> a, b = b, a
```

Dari contoh diatas, dapat dilihat bahwa kedua statemnt merupakan tuple. Bagian kiri merupakan tuple dari variable dan bagian kanan merupakan tuple dari expresions. Tiap nilai pada bagian kanan diberikan/ditugaskan ke masing-masing variable di sebelah kiri. Semua expresions pada bagian kiri dievaluasi sebelum diberikan penugasan.

Yang perlu diperhatikan adalah jumlah variabel antara sisi kiri dan sisi kanan harus sama.

```
>>> a, b = 1, 2, 3
ValueError: too many values to unpack
```

Pada sisi sebelah kanan biasanya terdapat data sekuensial string, list atau tuple. Sebagai contoh, kita akan membagi alamat email menjadi user name dan domain seperti berikut ini.

```
>>> email = 'didanendya@ti.ukdw.ac.id'
>>> username, domain = email.split('@')
```

Nilai yang dikembalikan dari **split** terdiri dari dua elemen yang dipisahkan tanda "@", elemen pertama berisi **username** dan elemen selanjutnya berisi **domain**.

```
>>> print(username)
didanendya
>>> print(domain)
ti.ukdw.ac.id
```

13.1.4 Dictionaries and Tuple

Dictionaries mempunyai metode yang disebut items untuk mengembalikan nilai list dari tuple dimana tiap tuple-nya merupakan key-value pair (pasangan kunci dan nilai).

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('b', 1), ('a', 10), ('c', 22)]
```

Seperti pada dictionary umumnya, item yang dikembalikan nilainya tidak berurutan. Bagaimanapun juga, list dari tuple adalah list, dan tuple membandingkannya sehingga kita dapat melakukan pengurutan (sort) pada tuple. Melakukan konversi dictionary dari list tuple merupakan cara untuk menampilkan isi dictionary yang diurutkan berdasarkan kuncinya.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> t
[('b', 1), ('a', 10), ('c', 22)]
>>> t.sort()
>>> t
[('a', 10), ('b', 1), ('c', 22)]
```

List yang muncul diurutkan secara ascending berdasarkan alfabet dan key value.

13.1.5 Multipenugasan dengan dictionaries

Kombinasi antara items, tuple assignment dan for akan menghasilkan kode tertentu pada keys dan values dari dictionary dalam satu loop.

```
for key, val in list(d.items()):  
    print(val, key)
```

Pada bagian looping ini, terdapat 2 iterasi variabel karena item mengembalikan list dari tuple dan key. val merupakan penugasan tuple yang melakukan iterasi berulang melalui pasangan key-value pada dictionary. Pada setiap iterasi yang melalui loop, baik key dan value akan bergerak maju menuju pasangan key-value berikutnya pada dictionary (masih dalam urutan hash). Output dari looping diatas

```
10 a  
22 c  
1 b
```

Jika dilakukan penggabungan dari kedua teknik diatas, kita dapat melakukan pencetakan isi dictionary yang diurutkan berdasarkan nilai yang disimpan di setiap pasangan key-value. Untuk melakukannya langkah pertama dengan membuat list tuple di mana masing-masing tuple beranggotakan (value,key). Method item akan memberikan list tupel (value,key) dan akan melakukan pengurutan berdasarkan value, bukan key. Setelah daftar list dengan value-key tuple terbentuk, langkah selanjutnya mengurutkan list tersebut dengan urutan terbalik dan melakukan pencetakan list baru yang disortir

```
>>> d = {'a':10, 'b':1, 'c':22}  
>>> l = list()  
>>> for key, val in d.items() :  
...     l.append( (val, key) )  
>>> l  
[(10, 'a'), (22, 'c'), (1, 'b')]  
>>> l.sort(reverse=True)  
>>> l  
[(22, 'c'), (10, 'a'), (1, 'b')]
```

Dengan melakukan penyusunan list dari tuple yang memiliki value sebagai elemen pertama, akan mudah untuk mengurutkan list dari tuple tersebut dan mendapatkan isi dictionary yang diurutkan berdasarkan nilainya.

13.1.6 Kata yang sering muncul

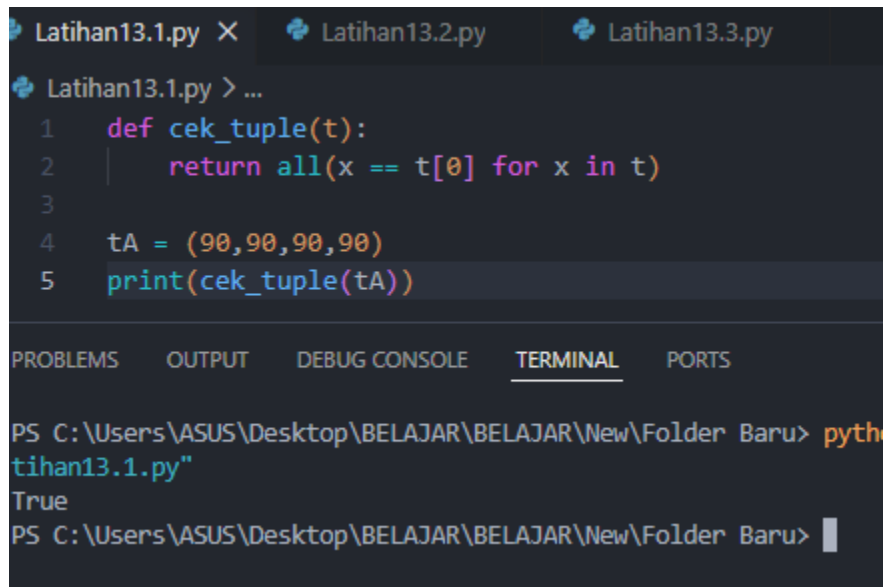
Pada bagian ini kita akan mencoba menampilkan kata yang sering muncul pada text dari Romeo and Juliet Act 2, Scene 2. Silakan download romeo-full.txt pada bagian materi. Kita akan coba membuat program menggunakan list dari tuple untuk menampilkan 10 kata yang paling sering muncul.

Penguraian dan analisis data yang kompleks ini dapat dilakukan dengan 19 baris program python dipahami. Hal tersebut merupakan salah satu alasan mengapa python adalah pilihan yang baik sebagai bahasa untuk mengeksplorasi informasi.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Latihan 13.1



```
Latihan13.1.py X  Latihan13.2.py  Latihan13.3.py
Latihan13.1.py > ...
1  def cek_tuple(t):
2      return all(x == t[0] for x in t)
3
4  tA = (90,90,90,90)
5  print(cek_tuple(tA))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Folder Baru> pyth
atihan13.1.py"
True
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Folder Baru> |
```

Program di atas +menggunakan fungsi all() untuk mengecek apakah semua isi tuple sama dengan elemen pertama. Jika semua sama maka output True, jika ada yang berbeda maka output False.

Latihan 13.2

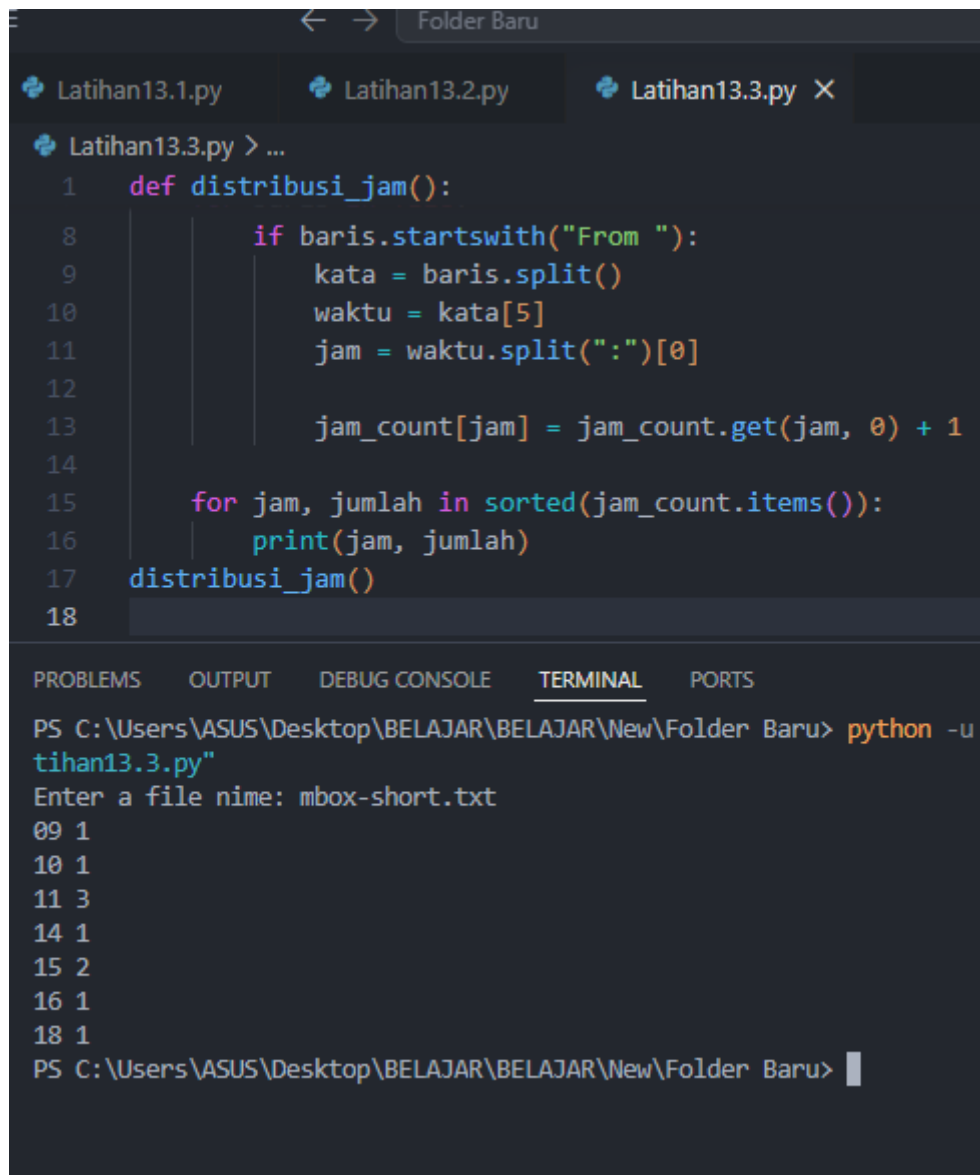
```
Latihan13.1.py  Latihan13.2.py X  Latihan13.3.py
Latihan13.2.py > ...
1  def biodata():
2      data = ('Jendri M Japen', '71251211', 'Yogyakarta')
3
4      print("NIM: ", data[1])
5      print("NAMA: ", data[0])
6      print("ALAMAT: ", data[2])
7
8      print("NIM: ", tuple(data[1]))
9
10     nama_depan = data[0].split()[0]
11     print("NAMA DEPAN: ", tuple(nama_depan[1:]))
12
13     nama_tabola_bale = tuple(reversed(data[0].split()))
14     print("NAMA TERBALIK: ", nama_tabola_bale)
15
16     biodata()
    ,

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Folder Baru> python -u "c:\U
tihan13.2.py"
NIM: 71251211
NAMA: Jendri M Japen
ALAMAT: Yogyakarta
NIM: ('7', '1', '2', '5', '1', '2', '1', '1')
NAMA DEPAN: ('e', 'n', 'd', 'r', 'i')
NAMA TERBALIK: ('Japen', 'M', 'Jendri')
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Folder Baru> 
```

Program di atas menggunakan tuple untuk menyimpan data diri seperti nama, NIM, dan alamat. Fungsi def digunakan agar program lebih rapi dan mudah dipanggil kembali. Program juga memproses data tuple untuk menampilkan NIM dalam bentuk tuple karakter, nama depan, dan nama terbalik.

Latihan 13.3



```
Folder Baru

Latihan13.1.py  Latihan13.2.py  Latihan13.3.py X

Latihan13.3.py > ...
1  def distribusi_jam():
8      if baris.startswith("From "):
9          kata = baris.split()
10         waktu = kata[5]
11         jam = waktu.split(":")[0]
12
13         jam_count[jam] = jam_count.get(jam, 0) + 1
14
15     for jam, jumlah in sorted(jam_count.items()):
16         print(jam, jumlah)
17 distribusi_jam()
18

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Folder Baru> python -u Latihan13.3.py
Enter a file name: mbox-short.txt
09 1
10 1
11 3
14 1
15 2
16 1
18 1
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Folder Baru>
```

Program di atas menggunakan dictionary untuk menghitung jumlah email berdasarkan jam pengiriman. Data jam diambil dari file mbox-short.txt, lalu dihitung kemunculannya dan ditampilkan secaraurut.

Link github: <https://github.com/japenjendri-design/Praktikkum-Algoritma-dan-Pemrograman.git>